

Team 2 – Asset Monitoring & Tracking

Task: Asset Inventory Foundation

Deliverables:

- PostgreSQL database schema
 - Asset inventory data model
-

Asset Inventory Foundation Structure:

Team2/

— database/

— postgres.py

— init_db.py

— models/

— asset.py

— vulnerability.py

— open_port.py

— dns_record.py

— scan_snapshot.py

— asset_change.py

—asset_log.py

Install Environment:

In kali linux Install postgresql,

sudo apt install postgresql

```
kali@kali: ~/Team2
Session Actions Edit View Help

(kali@kali)-[~/Team2]
$ sudo apt install postgresql
The following packages were automatically installed and are no longer required:
enchanted-2 libipc-run-perl linux-image-6.16.8+kali-amd64
libavc1394-0 libpostal-data openjdk-21-jre
libcrypt-dev libpostal1 openjdk-21-jre-headless
libgdk-pixbuf2.0-bin libraw23t64 postgresql-common-dev
libio-pty-perl libsframe2
Use 'sudo apt autoremove' to remove them.

Upgrading:
postgresql postgresql-common
postgresql-client-common postgresql-common-dev

Summary:
Upgrading: 4, Installing: 0, Removing: 0, Not Upgrading: 1035
Download size: 0 B / 223 kB
Freed space: 28.7 kB

Continue? [Y/n] y
Preconfiguring packages ...
(Reading database... 575676 files and directories currently installed.)
Preparing to unpack .../postgresql_18+290_all.deb...
Progress: [ 6%]
```

create python environment:

```
python -m venv venv
```

```
source venv/bin/activate
```

```
Setting up postgresql-client-common (290)...
Setting up postgresql-common (290)...
Installing new version of config file /etc/postgresql-common/pg_upgradecluster.d/analyze...
postgresql.service is a disabled or a static unit, not starting it.
Setting up postgresql (18+290)...
Setting up postgresql-common-dev (290)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.1.3)...

(kali@kali)-[~/Team2]
$ python -m venv venv
source venv/bin/activate

(venv)-(kali@kali)-[~/Team2]
$
```

Install packages:

```
pip install sqlalchemy
```

```
pip install psycopg2-binary
```

```
pip install python-dotenv
```

```
Session Actions Edit View Help
source venv/bin/activate

(venv)-(kali@kali)-[~/Team2]
└─$ pip install sqlalchemy
pip install psycopg2-binary
pip install python-dotenv
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.50-cp313-cp313-manylinux2014_x86_64.manylinux_2_
17_x86_64.manylinux_2_28_x86_64.whl.metadata (9.5 kB)
Collecting greenlet≥1 (from sqlalchemy)
  Downloading greenlet-3.5.1-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28
_x86_64.whl.metadata (3.8 kB)
Collecting typing-extensions≥4.6.0 (from sqlalchemy)
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Downloading sqlalchemy-2.0.50-cp313-cp313-manylinux2014_x86_64.manylinux_2_17
_x86_64.manylinux_2_28_x86_64.whl (3.3 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.3/3.3 MB 6.3 MB/s 0:00:00
Downloading greenlet-3.5.1-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x
86_64.whl (613 kB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 613.9/613.9 kB 6.6 MB/s 0:00:00
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Installing collected packages: typing-extensions, greenlet, sqlalchemy
Successfully installed greenlet-3.5.1 sqlalchemy-2.0.50 typing-extensions-4.1
5.0
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.12-cp313-cp313-manylinux2014_x86_64.manylin
ux_2_17_x86_64.whl.metadata (4.9 kB)
Downloading psycopg2_binary-2.9.12-cp313-cp313-manylinux2014_x86_64.manylinux
_2_17_x86_64.whl (4.3 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.3/4.3 MB 4.7 MB/s 0:00:01
Installing collected packages: psycopg2-binary
Successfully installed psycopg2-binary-2.9.12
Collecting python-dotenv
  Downloading python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
```

create database:

Check PostgreSQL Status and Make sure PostgreSQL is running.

```
sudo systemctl status postgresql
```

If not running:

```
sudo systemctl start postgresql
```

Enable it on boot:

```
sudo systemctl enable postgresql
```

Login to PostgreSQL and Switch to the PostgreSQL user:

```
sudo -i -u postgres
```

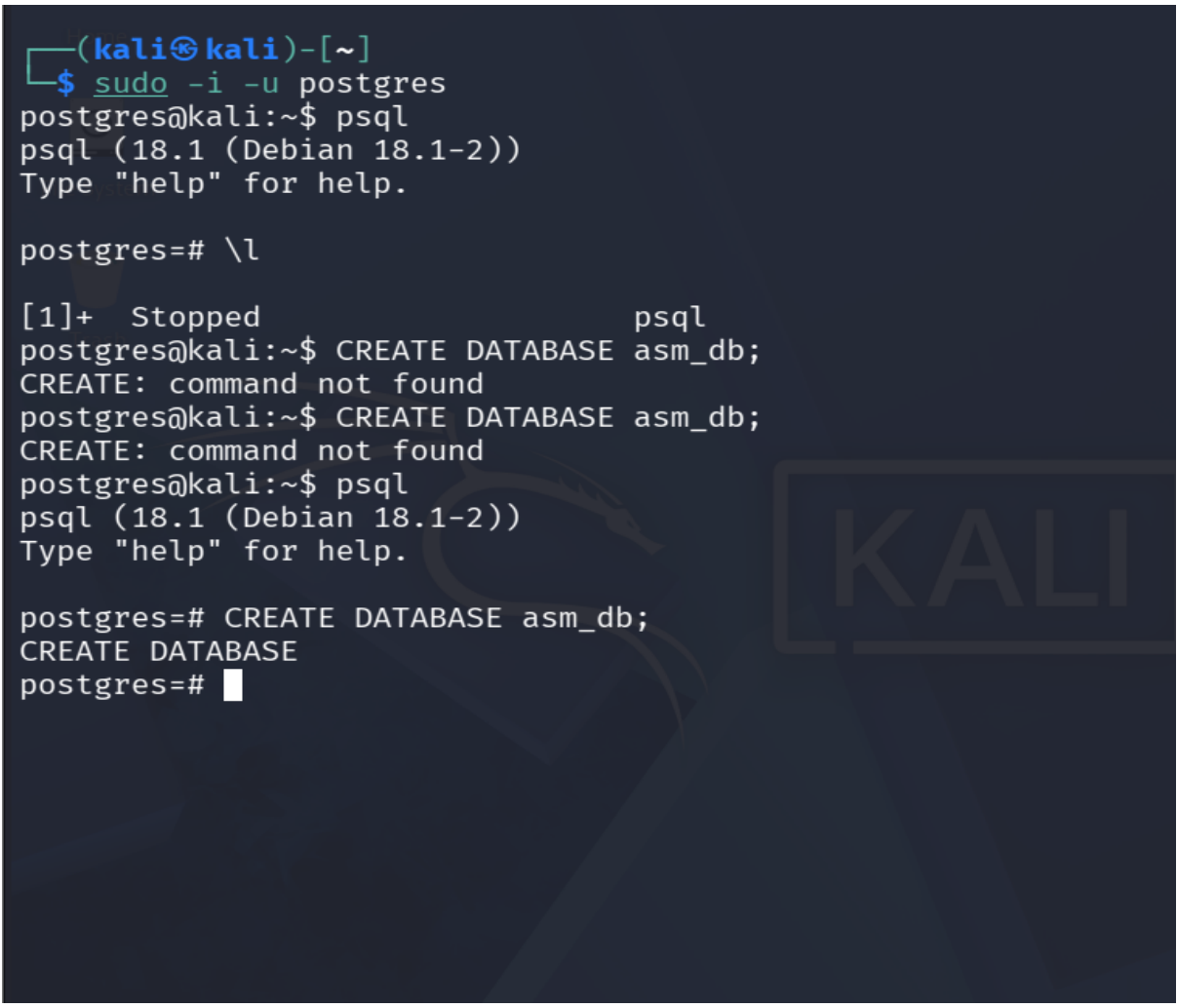
Open PostgreSQL shell:

```
psql
```

Create Project Database

Create the database for the Asset Monitoring System:

```
CREATE DATABASE asm_db;
```



```
(kali㉿kali)-[~]  
$ sudo -i -u postgres  
postgres@kali:~$ psql  
psql (18.1 (Debian 18.1-2))  
Type "help" for help.  
  
postgres=# \l  
  
[1]+  Stopped                  psql  
postgres@kali:~$ CREATE DATABASE asm_db;  
CREATE: command not found  
postgres@kali:~$ CREATE DATABASE asm_db;  
CREATE: command not found  
postgres@kali:~$ psql  
psql (18.1 (Debian 18.1-2))  
Type "help" for help.  
  
postgres=# CREATE DATABASE asm_db;  
CREATE DATABASE  
postgres=#
```

The screenshot shows a terminal window on a Kali Linux system. The user switches to the postgres user and enters the psql shell. They attempt to create a database named 'asm_db' but receive 'command not found' errors. The terminal background features a Kali Linux logo and the word 'KALI' in large letters.

Verify Database Creation

List databases again:

\l

List of databases							
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	
ICU Rules	Access privileges						
-----+-----+-----+-----+-----+-----+-----							
asm_db	postgres	UTF8	libc	en_IN	en_IN		
gvmd	_gvm	UTF8	libc	en_IN	en_IN		
postgres	postgres	UTF8	libc	en_IN	en_IN		
template0	postgres	UTF8	libc	en_IN	en_IN		
	=c/postgres		+				
	postgres=CTc/postgres						
template1	postgres	UTF8	libc	en_IN	en_IN		
	=c/postgres		+				
	postgres=CTc/postgres						
(5 rows)							
(END)							

Connect to the New Database

\c asm_db

```
[1]+ Stopped psql
postgres@kali:~$ CREATE DATABASE asm_db;
CREATE: command not found
postgres@kali:~$ CREATE DATABASE asm_db;
CREATE: command not found
postgres@kali:~$ psql
psql (18.1 (Debian 18.1-2))
Type "help" for help.

postgres=# CREATE DATABASE asm_db;
CREATE DATABASE
postgres=# \l

[2]+ Stopped psql
postgres@kali:~$ psql
psql (18.1 (Debian 18.1-2))
Type "help" for help.

postgres=# \c asm_db
You are now connected to database "asm_db" as user "postgres".
asm_db=#
```

Verify Current Connection

```
SELECT current_database();
```

```
postgres=# \c asm_db
You are now connected to database "asm_db" as user "postgres".
asm_db=# SELECT current_database();
 current_database
-----
asm_db
(1 row)

asm_db=#
```

Create Dedicated Application User :

```
CREATE USER asm_user WITH PASSWORD 'asmuser';
```

Grant Database Permissions

```
GRANT ALL PRIVILEGES ON DATABASE asm_db TO asm_user;
```

```
postgres=# \c asm_db
You are now connected to database "asm_db" as user "postgres".
asm_db=# SELECT current_database();
 current_database
-----
asm_db
(1 row)

asm_db=# CREATE USER asm_user WITH PASSWORD 'asmuser';
CREATE ROLE
asm_db=# GRANT ALL PRIVILEGES ON DATABASE asm_db TO asm_user;
GRANT
asm_db=#
```

Verify User Creation

```
\du
```

```
asm_db=# CREATE USER asm_user WITH PASSWORD 'asmuser';
CREATE ROLE
asm_db=# GRANT ALL PRIVILEGES ON DATABASE asm_db TO asm_user;
GRANT
asm_db=# \du

              List of roles
-----
Role name | Attributes
-----+-----
_gvm      |
asm_user  |
dba       | Superuser, No inheritance, Cannot login
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS

asm_db=#
```

PostgreSQL Permissions

Login as postgres:

```
sudo -u postgres psql
```

Connect to your database:

```
\c asm_db
```

Grant schema permissions:

```
GRANT ALL ON SCHEMA public TO asm_user;
```

Also make asm_user the owner of the schema:

```
ALTER SCHEMA public OWNER TO asm_user;
```

Give database privileges:

```
GRANT ALL PRIVILEGES ON DATABASE asm_db TO asm_user;
```

Verify,

```
\l
```

```
(kali㉿kali)-[~/Team2]
└─$ sudo -u postgres psql
psql (18.1 (Debian 18.1-2))
Type "help" for help.

postgres=# \c asm_db
You are now connected to database "asm_db" as user "postgres".
asm_db=# GRANT ALL ON SCHEMA public TO asm_user;
GRANT
asm_db=# ALTER SCHEMA public OWNER TO asm_user;
ALTER SCHEMA
asm_db=# GRANT ALL PRIVILEGES ON DATABASE asm_db TO asm_user;
GRANT
asm_db=#
```

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
asm_db	postgres	UTF8	libc	en_IN	en_IN			=Tc/postgres + postgres=Ctc/postgres+ asm_user=Ctc/postgres
gvmd	_gvm	UTF8	libc	en_IN	en_IN			
postgres	postgres	UTF8	libc	en_IN	en_IN			=c/postgres + postgres=Ctc/postgres
template0	postgres	UTF8	libc	en_IN	en_IN			=c/postgres + postgres=Ctc/postgres
template1	postgres	UTF8	libc	en_IN	en_IN			=c/postgres + postgres=Ctc/postgres
(5 rows)								
~								
~								
~								

```
— database/
  |
  — postgres.py
  — init_db.py
```

Database:

Create database directory,

```
mkdir database
```

then inside that directory create 2 files,

```
nano postgres.py
```

```
nano init_db.py
```

these 2 files are the foundation of the database layer. It make sure SQLAlchemy knows how to connect to PostgreSQL and how to create all tables automatically.

postgres.py:

Purpose:

This file contains the PostgreSQL database configuration and initializes the SQLAlchemy engine. It defines the database connection parameters such as host, port, database name, username, and password. It also creates the SQLAlchemy Base class, which serves as the parent class for all database models.

Responsibilities:

- Store database connection settings.
- Create the SQLAlchemy database engine.
- Define the common base class for all models.

init_db.py

Purpose:

This file is responsible for initializing the database schema. It imports all model classes and creates the corresponding tables in PostgreSQL using SQLAlchemy's metadata management.

Responsibilities:

- Register all database models.
- Create database tables automatically.
- Initialize the database schema during project setup.

Model :

Model structure is

```
model/
├── asset.py
├── vulnerability.py
├── open_port.py
├── dns_record.py
├── scan_snapshot.py
├── asset_change.py
└── asset_log.py
```

Create model directory,

```
mkdir model
```

then create files,

```
nano asset.py
```

```
nano vulnerability.py
```

```
nano open_port.py
```

```
nano dns_record.py
```

```
nano scan_snapshot.py
```

```
nano asset_change.py
```

```
nano asset_log.py
```

The Models layer defines the database schema of the Asset Monitoring and Change Detection system using SQLAlchemy ORM. Each model represents a database table and specifies its fields, relationships, and constraints. The models provide a structured way to store and retrieve asset information, vulnerabilities, scan results, detected changes, and alert logs.

asset.py:

Represents the central asset inventory table. It stores information about monitored assets such as domain name, IP address, asset type, and first scan timestamp. This model acts as the parent entity and maintains relationships with vulnerabilities, open ports, DNS records, snapshots, detected changes, and alert logs.

vulnerability.py

Stores vulnerability information discovered during security scans. Each vulnerability is linked to a specific asset and contains details such as CVE identifier, severity level, CVSS score, remediation guidance, and scanner template information.

open_port.py

Stores information about open network ports detected on an asset. The model records the port number, protocol, service name, and service banner associated with the discovered port.

dns_record.py

Stores DNS records associated with an asset. It supports different DNS record types such as A, AAAA, MX, TXT, and CNAME, allowing the system to monitor DNS-related changes over time.

scan_snapshot.py

Stores timestamped snapshots of scanner outputs. Each snapshot represents the state of an asset at a particular point in time and serves as the basis for change detection and historical analysis.

asset_change.py

Stores changes detected between previous and current asset snapshots. It records the type of change, old value, new value, risk score, and detection timestamp, enabling the system to track security-relevant modifications.

asset_log.py

Stores notification and alert delivery logs. It records the notification channel, delivery status, and timestamp, providing an audit trail for alerting activities.

To create table ,

```
cd Team2
```

```
python -m database.init_db
```

```
sudo -u postgres psql asm_db
```

```
\dt
```

```
Session Actions Edit View Help
asm_db=# \q

(kali@kali)-[~/Team2]
$ sudo -u postgres psql
psql (18.1 (Debian 18.1-2))
Type "help" for help.

postgres=# \l

zsh: suspended  sudo -u postgres psql

(kali@kali)-[~/Team2]
$ python -m database.init_db
All tables created successfully!

(kali@kali)-[~/Team2]
$ sudo -u postgres psql asm_db
psql (18.1 (Debian 18.1-2))
Type "help" for help.

asm_db=# \dt
          List of tables
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | asset_changes  | table | asm_user
public | asset_logs     | table | asm_user
public | assets         | table | asm_user
public | dns_records    | table | asm_user
public | open_ports     | table | asm_user
public | scan_snapshots | table | asm_user
public | vulnerabilities | table | asm_user
(7 rows)

asm_db=#
```

This screenshot shows the list of all tables

To display the structure of tables:

/d assets

```
Session Actions Edit View Help
public | asset_changes | table | asm_user
public | asset_logs | table | asm_user
public | assets | table | asm_user
public | dns_records | table | asm_user
public | open_ports | table | asm_user
public | scan_snapshots | table | asm_user
public | vulnerabilities | table | asm_user
(7 rows)

asm_db=# \d assets
Table "public.assets"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
id            | integer        |           | not null | nextval('assets_id_seq'::regclass)
domain       | character varying |           | not null |
ip_address   | character varying |           | not null |
asset_type   | character varying |           | not null |
first_scan   | timestamp without time zone |           |
last_seen    | timestamp without time zone |           |
is_internal  | boolean        |           | false   |
is_external  | boolean        |           | true    |

Indexes:
  "assets_pkey" PRIMARY KEY, btree (id)
  "ix_assets_id" btree (id)
Referenced by:
  TABLE "asset_changes" CONSTRAINT "asset_changes_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
  TABLE "asset_logs" CONSTRAINT "asset_logs_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
  TABLE "dns_records" CONSTRAINT "dns_records_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
  TABLE "open_ports" CONSTRAINT "open_ports_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
  TABLE "scan_snapshots" CONSTRAINT "scan_snapshots_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
  TABLE "vulnerabilities" CONSTRAINT "vulnerabilities_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
```

This shows the structure of assets table

/d vulnerabilities

```
TABLE "scan_snapshots" CONSTRAINT "scan_snapshots_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
TABLE "vulnerabilities" CONSTRAINT "vulnerabilities_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)

asm_db=# \d vulnerabilities
Table "public.vulnerabilities"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
id            | integer        |           | not null | nextval('vulnerabilities_id_seq'::regclass)
asset_id      | integer        |           | not null |
cve_id        | character varying |           | not null |
severity      | character varying |           | not null |
cvss_score    | double precision |           |
template_id   | character varying |           |
remediation   | text           |           |

Indexes:
  "vulnerabilities_pkey" PRIMARY KEY, btree (id)
  "ix_vulnerabilities_id" btree (id)
Foreign-key constraints:
  "vulnerabilities_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)

asm_db=#
```

This shows the structure of vulnerabilities table

\d open_ports

```
asm_db=# \d open_ports
Table "public.open_ports"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
id            | integer        |           | not null | nextval('open_ports_id_seq'::regclass)
asset_id      | integer        |           | not null |
port          | integer        |           | not null |
protocol      | character varying |           | not null |
service_name  | character varying |           |
banner        | text           |           |

Indexes:
  "open_ports_pkey" PRIMARY KEY, btree (id)
  "ix_open_ports_id" btree (id)
Foreign-key constraints:
  "open_ports_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)

asm_db=#
```

This shows the structure of open_ports table

\d dns_records

```
asm_db=# \d dns_records
Table "public.dns_records"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 id           | integer        |           | not null | nextval('dns_records_id_seq'::regclass)
 asset_id     | integer        |           | not null |
 record_type  | character varying |           | not null |
 value       | text           |           | not null |
Indexes:
    "dns_records_pkey" PRIMARY KEY, btree (id)
    "ix_dns_records_id" btree (id)
Foreign-key constraints:
    "dns_records_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
asm_db=#
```

This shows the structure of dns records table

\d asset_changes

```
asm_db=# \d asset_changes
Table "public.asset_changes"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 id           | integer        |           | not null | nextval('asset_changes_id_seq'::regclass)
 asset_id     | integer        |           | not null |
 change_type  | character varying |           | not null |
 old_value    | text           |           |
 new_value    | text           |           |
 risk_score   | double precision |           |
 detected_at  | timestamp without time zone |           |
Indexes:
    "asset_changes_pkey" PRIMARY KEY, btree (id)
    "ix_asset_changes_id" btree (id)
Foreign-key constraints:
    "asset_changes_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
asm_db=#
```

This shows the structure of asset_changes table

\d asset_logs

```
asm_db=# \d asset_logs
Table "public.asset_logs"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 id           | integer        |           | not null | nextval('asset_logs_id_seq'::regclass)
 asset_id     | integer        |           | not null |
 channel      | character varying |           | not null |
 status       | character varying |           | not null |
 sent_at      | timestamp without time zone |           |
Indexes:
    "asset_logs_pkey" PRIMARY KEY, btree (id)
    "ix_asset_logs_id" btree (id)
Foreign-key constraints:
    "asset_logs_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
asm_db=#
```

This shows the structure of asset_logs table

\d scan_snapshots

```
asm_db=# \d scan_snapshots
Table "public.scan_snapshots"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 id           | integer        |           | not null | nextval('scan_snapshots_id_seq'::regclass)
 asset_id     | integer        |           | not null |
 scanner      | character varying |           | not null |
 scanned_at   | timestamp without time zone |           |
 raw_json     | text           |           | not null |
Indexes:
    "scan_snapshots_pkey" PRIMARY KEY, btree (id)
    "ix_scan_snapshots_id" btree (id)
Foreign-key constraints:
    "scan_snapshots_asset_id_fkey" FOREIGN KEY (asset_id) REFERENCES assets(id)
asm_db=#
```

This shows the structure of scan_snapshots table

Create test_data.py file that inserts data into all tables to test working of database

nano test_data.py

then run python test_data.py

```
TABLE vulnerabilities CONSTRAINT vulnerabilities_asset_id_fkey FOREIGN KEY (asset_id) REFERENCES assets(id)
asm_db=# \q
(kali@kali)-[~/Team2]
$ ls
database model venv
(kali@kali)-[~/Team2]
$ nano test_data.py
(kali@kali)-[~/Team2]
$ python test_data.py
/home/kali/Team2/test_data.py:24: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  first_scan=datetime.utcnow(),
/home/kali/Team2/test_data.py:25: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  last_seen=datetime.utcnow(),
Asset Created: 1
All test data inserted successfully!
(kali@kali)-[~/Team2]
```

Created the test_data file to test working of database and all test data is inserted successfully.

Check data in database,

sudo -u postgres psql asm_db

SELECT*FROM assets;

```
(kali@kali)-[~/Team2]
$ sudo -u postgres psql asm_db
psql (18.1 (Debian 18.1-2))
Type "help" for help.

asm_db=# SELECT*FROM assets;
 id | domain | ip_address | asset_type | first_scan | last_seen | is_internal | is_external
----+-----+-----+-----+-----+-----+-----+-----
  1 | example.com | 192.168.1.10 | server | 2026-06-15 01:13:28.26194 | 2026-06-15 01:13:28.26197 | t | f
(1 row)

asm_db=#
```

This displays the inserted data in the assets table

SELECT*FROM vulnerabilities;

```
(kali@kali)-[~/Team2]
$ sudo -u postgres psql asm_db
psql (18.1 (Debian 18.1-2))
Type "help" for help.

asm_db=# SELECT*FROM assets;
 id | domain | ip_address | asset_type | first_scan | last_seen | is_internal | is_external
----+-----+-----+-----+-----+-----+-----+-----
  1 | example.com | 192.168.1.10 | server | 2026-06-15 01:13:28.26194 | 2026-06-15 01:13:28.26197 | t | f
(1 row)

asm_db=# SELECT*FROM vulnerabilities;
 id | asset_id | cve_id | severity | cvss_score | template_id | remediation
----+-----+-----+-----+-----+-----+-----
  1 | 1 | CVE-2025-1234 | High | 8.5 | temp-001 | Update software
(1 row)

asm_db=#
```

This displays the inserted data in the vulnerabilities table

SELECT*FROM open_ports;

```
(kali@kali) - [~/Team2]
$ sudo -u postgres psql asm_db
psql (18.1 (Debian 18.1-2))
Type "help" for help.

asm_db=# SELECT*FROM assets;
 id | domain | ip_address | asset_type | first_scan | last_seen | is_internal | is_external
-----+-----+-----+-----+-----+-----+-----+-----
  1 | example.com | 192.168.1.10 | server | 2026-06-15 01:13:28.26194 | 2026-06-15 01:13:28.26197 | t | f
(1 row)

asm_db=# SELECT*FROM vulnerabilities;
 id | asset_id | cve_id | severity | cvss_score | template_id | remediation
-----+-----+-----+-----+-----+-----+-----
  1 | 1 | CVE-2025-1234 | High | 8.5 | temp-001 | Update software
(1 row)

asm_db=# SELECT*FROM open_ports;
 id | asset_id | port | protocol | service_name | banner
-----+-----+-----+-----+-----+-----
  1 | 1 | 22 | TCP | SSH | OpenSSH 9.0
(1 row)

asm_db=#
```

This displays the inserted data in the open_ports table

SELECT*FROM dns_records;

```
asm_db=# SELECT*FROM dns_records;
 id | asset_id | record_type | value
-----+-----+-----+-----
  1 | 1 | A | 192.168.1.10
(1 row)

asm_db=#
```

This displays the inserted data in the dns_records table

SELECT*FROM asset_changes;

```
asm_db=# SELECT*FROM dns_records;
 id | asset_id | record_type | value
-----+-----+-----+-----
  1 | 1 | A | 192.168.1.10
(1 row)

asm_db=# SELECT*FROM asset_changes;
 id | asset_id | change_type | old_value | new_value | risk_score | detected_at
-----+-----+-----+-----+-----+-----+-----
  1 | 1 | Port Added | 80 | 443 | 7.5 | 2026-06-15 01:13:28.32455
(1 row)

asm_db=#
```

This displays the inserted data in the asset_changes table

SELECT*FROM asset_logs;

```
asm_db=# SELECT*FROM dns_records;
 id | asset_id | record_type | value
-----+-----+-----+-----
  1 | 1 | A | 192.168.1.10
(1 row)

asm_db=# SELECT*FROM asset_changes;
 id | asset_id | change_type | old_value | new_value | risk_score | detected_at
-----+-----+-----+-----+-----+-----+-----
  1 | 1 | Port Added | 80 | 443 | 7.5 | 2026-06-15 01:13:28.32455
(1 row)

asm_db=# SELECT*FROM asset_logs;
 id | asset_id | channel | status | sent_at
-----+-----+-----+-----+-----
  1 | 1 | email | success | 2026-06-15 01:13:28.332868
(1 row)

asm_db=#
```

This displays the inserted data in the asset_logs table

```
SELECT*FROM scan_snapshots;
```

```
asm_db=# SELECT*FROM dns_records;
 id | asset_id | record_type | value
-----+-----+-----+-----
  1 |      1 | A           | 192.168.1.10
(1 row)

asm_db=# SELECT*FROM asset_changes;
 id | asset_id | change_type | old_value | new_value | risk_score | detected_at
-----+-----+-----+-----+-----+-----+-----
  1 |      1 | Port Added | 80        | 443       | 7.5        | 2026-06-15 01:13:28.32455
(1 row)

asm_db=# SELECT*FROM asset_logs;
 id | asset_id | channel | status | sent_at
-----+-----+-----+-----+-----
  1 |      1 | email   | success | 2026-06-15 01:13:28.332868
(1 row)

asm_db=# SELECT*FROM scan_snapshots;
 id | asset_id | scanner | scanned_at | raw_json
-----+-----+-----+-----+-----
  1 |      1 | Nmap    | 2026-06-15 01:13:28.350306 | {"ports":[22,80,443]}
(1 row)

asm_db=#
```

This displays the inserted data in the scan_snapshots table